



CS4002 Applied Programming

National University of Computer & Emerging
Sciences
Islamabad, Pakistan

Ms. Umarah Qaseem



1

Object Oriented Programming - III

3

3

Default copy constructor

- A type of **constructor** that is used to **initialize an object with another object of the same type** is known as **default copy constructor**.
- It is **by default available in all classes**
- syntax is **ClassName(ClassName &Variable)**

5

Default Copy Constructor - Example

```
class Distance {
private:
    int feet; float inches;
public:
    Distance( ):feet(0),inches(0.0)
    { }
    Distance(int ft, float in): feet(ft), inches(in)
    { }
    Distance(Distance &d)
    {
        feet = d.feet; inches = d.inches; }
    void showDistance()
    {
        cout<<"Feet = "<<feet;
        cout<<"Inches="<<inches;
    }
};
```

```
void main() {
    Distance dist1(11,22.8);
    Distance dist2(dist1);
    Distance dist3 = dist1;
    dist1.showDistance();
    dist2.showDistance();
    dist3.showDistance();
}
```

6

Memberwise Copy

- Another different format has exactly the same effect, causing object to be copied member-by-member into other object.
- Remember, ***object1 = object2*** will ***not invoke the copy constructor***. However, **this statement is legal** and **simply assigns the values of object2 to object1, member-by-member**.
- ***This is the task of the overloaded assignment operator (=).***

7

Memberwise Copy - Example

```
class Distance
{
    private:
        int feet; float inches;
    public:
        Distance():feet(0),inches(0.0)
        { }
        Distance(int ft, float in): feet(ft), inches(in)
        { }
        void showDistance()
        {
            cout<<"Feet = "<<feet;
            cout<<"Inches="<<inches;
        }
};

void main()
{
    Distance dist1(11,22.8);
    Distance dist2(dist1);
    Distance dist3 = dist1;
    Distance dist4;
    dist4 = dist1;
    dist1.showDistance();
    dist2.showDistance();
    dist3.showDistance();
    dist4.showDistance();
}
```

8

Returning Objects from Function - Example

```

class Distance {
public:
    int feet;
    Distance():feet(0)
    { }
    Distance(int ft): feet(ft)
    { }
    Distance addDist(Distance d)
    {
        Distance temp;
        temp.feet = this->feet + d.feet;
        return temp;
    }
    void showDistance()
    { cout<<"Feet = "<<feet; }
};

```

```

void main()
{
    Distance dist1, dist3;
    Distance dist2(11);
    dist3 = dist1.addDist(dist2);
    dist1.showDistance();
    dist2.showDistance();
    dist3.showDistance();
}

```

The diagram illustrates the execution of the `addDist` method. A red arrow originates from the line `dist3 = dist1.addDist(dist2);` in the `main` function and points to the `addDist` method definition within the `Distance` class. A second red arrow points from the `return temp;` statement inside the `addDist` method back to the assignment operation in `main`, indicating the return of the object.

9

The Scope of Variables

Global variables are declared outside all functions and are accessible to all functions in its scope. The scope of a global variable starts from its declaration and continues to the end of the program.

Local variables are defined inside functions. The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable.

10

The Scope of Variables

The **data fields** are declared as **variables** and are **accessible to all constructors and functions in the class**.

Data fields and functions can be declared in any order in a class.

For example, all the following declarations are the same:

```
class Circle
{
public:
    Circle();
    Circle(double);
    double getArea();
    double getRadius();
    void setRadius(double);

private:
    double radius;
};
```

(a)

```
class Circle
{
public:
    Circle();
    Circle(double);

private:
    double radius;

public:
    double getArea();
    double getRadius();
    void setRadius(double);
};
```

(b)

```
class Circle
{
private:
    double radius;

public:
    double getArea();
    double getRadius();
    void setRadius(double);

public:
    Circle();
    Circle(double);
};
```

(c)

11

Accessing Object Members via Pointers

- **Complete** and **Shorthand Member selection operator** for **accessing object members**. For example:

```
Circle circle1;
Circle *pCircle = &circle1;
cout << "The radius is " << (*pCircle).radius << endl;
cout << "The area is " << (*pCircle).getArea() << endl;
pCircle->radius = 5.5;
cout << "The radius is " << pCircle->radius << endl;
cout << "The area is " << pCircle->getArea() << endl;
```

13

Passing Objects to Functions

■ Three ways

1. **Pass objects by value**
2. **Pass objects by reference**
3. **Pass objects by reference via pointers**

14

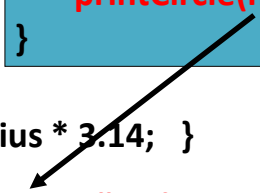
Pass by value

```

class Circle {
    private:
        double radius;
    public:
        Circle();
        Circle(double);
        double getArea();
};
Circle::Circle()
{
    radius = 1.0;
}
double Circle::getArea()
{
    return radius * radius * 3.14; }
void printCircle(Circle c)
{
    cout<<"the area = "<<c.getArea(); }
    
```

```

void main()
{
    Circle myCircle;
    printCircle(myCircle);
}
    
```



15

Pass by reference

```

class Circle {
    private:
        double radius;
    public:
        Circle();
        Circle(double);
        double getArea();
};
Circle::Circle()
{    radius = 1.0; }
double Circle::getArea()
{    return radius * radius * 3.14; }
void printCircle(Circle &c)
{    cout<<"the area = "<<c.getArea(); }

```

```

void main()
{
    Circle myCircle;
    printCircle(myCircle);
}

```

16

Pass by Reference via pointer

```

class Circle {
    private:
        double radius;
    public:
        Circle();
        Circle(double);
        double getArea();
};
Circle::Circle()
{    radius = 1.0; }
double Circle::getArea()
{    return radius * radius * 3.14; }
void printCircle(Circle *c)
{    cout<<"the area = "<<c->getArea(); }

```

```

void main()
{
    Circle myCircle;
    printCircle(&myCircle);
}

```

17

Copy Constructors

Each class may define **several overloaded constructors** and **one destructor**.

Additionally, every class has a **copy constructor**. The signature of the copy constructor is:

ClassName (ClassName &)

For example, the copy constructor for the Circle class is

Circle(Circle &)

The **copy constructor can be used to create an object initialized with another object's data.**

18

Shallow Copy vs. Deep Copy

The default copy constructor or assignment operator for copying objects performs a **shallow copy**, rather than a **deep copy**.

In shallow copy, if a data field in a class is a pointer, the address of the pointer is copied rather than its contents.

In deep copy, if a data field in a class is a pointer, the contents of that objects are copied rather than its reference.

19

Customizing Copy Constructor

- *The **default copy constructor or assignment operator** = performs a shallow copy.*
- To perform a **deep copy**, you have to implement the **copy constructor**.

20

Shallow Copy - Example

<pre> class Test { private: int i; int *p; public: Test(int x, int y) { i = x; p = new int(y); } void setVal(int x, int y) { i = x; *p = y; } void show() { cout<<"i = "<<i<<endl; cout<<"p = "<<*p<<endl; } }; </pre>	<pre> void main() { Test t1(5,10); Test t2(t1); t1.setVal(20,30); t1.show(); t2.show(); } Output: i = 20 p = 30 i = 5 p = 30 </pre>
--	--

21

Shallow Copy - Example

<pre> class Test{ private: int i; int *p; public: Test(int x, int y){ i = x; p = new int(y); } Test(Test &t) { i = t.i; p = t.p; } void setVal(int x, int y) { i = x; *p = y; } void show() { cout<<"i = "<<i<<endl; cout<<"p = "<<*p<<endl; } }; </pre>	<pre> void main() { Test t1(1,12); Test t2(t1); t1.setVal(0,70); t1.show(); t2.show(); } i = 0 p = 70 i = 1 p = 70 </pre>
--	--

22

Deep Copy - Example

<pre> class Test{ private: int i; int *p; public: Test(int x, int y){ i = x; p = new int(y); } Test(Test &t) { i = t.i; p = new int(*t.p); } void setVal(int x, int y) { i = x; *p = y; } void show() { cout<<"i = "<<i<<endl; cout<<"p = "<<*p<<endl; } }; </pre>	<pre> void main() { Test t1(5,1); Test t2(t1); t1.setVal(90,3); t1.show(); t2.show(); } i = 90 p = 3 i = 5 p = 1 </pre>
--	--

23

Deep Copy - Example

<pre> class Test { private: int i; int *p; public: Test(int x, int y):i(x), p(new int(y)) { } Test(Test &t):i(t.i), p(new int(*t.p)) { } void setVal(int x, int y) { i = x; *p = y; } void show() { cout<<"i = "<<i<<endl; cout<<"p = "<<*p<<endl; } }; </pre>	<pre> void main() { Test t1(5,10); Test t2(t1); t1.setVal(2,80); t1.show(); t2.show(); } </pre>
--	---

24

Shallow Copy of Dynamic Array - Example

<pre> class Test { public: int size; int *arr; Test(int size):size(size),arr(new int[size]) { for(int i=0; i<3; i++) arr[i] = 0; } Test(Test &t):size(t.size),arr(t.arr) { std::copy(t.arr, t.arr + size, arr); } void show() { for(int i=0; i<3; i++) cout<<arr[i]<<" "; } }; </pre>	<pre> void main() { Test t1(3); Test t2(t1); t1.arr[0] = 31; t1.show(); t2.show(); } </pre> 31 0 0 31 0 0
--	--

25

Deep Copy of Dynamic Array - Example

```

class Test
{ public:
    int size;
    int *arr;

    Test(int size):size(size),arr(new int[size])
    {
        for(int i=0; i<3; i++)
            arr[i] = 0;
    }
    Test(Test &t):size(t.size), arr(new int[size])
    {
        std::copy(t.arr, t.arr + size, arr);
    }
    void show()
    {
        for(int i=0; i<3; i++)
            cout<<arr[i]<<" ";
    }
};

void main()
{
    Test t1(3);
    Test t2(t1);

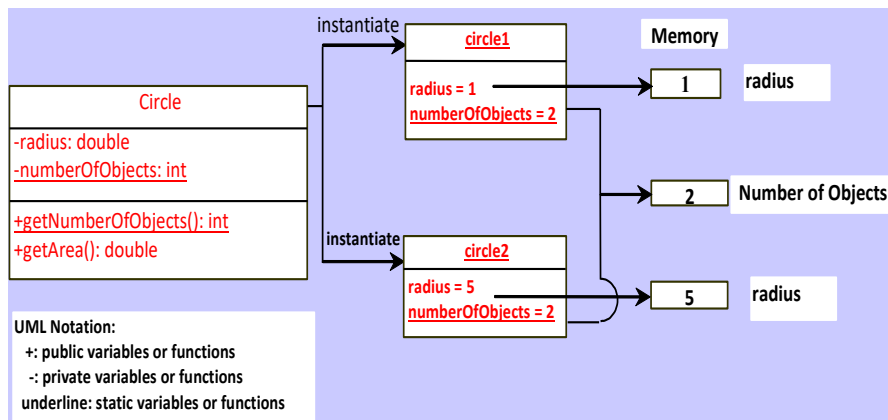
    t1.arr[0] = 31;

    t1.show();
    t2.show();
}

```

26

Instance and Static Members



30

Example – Non-Static data members

<pre>class Test{ private: int count; public: Test() { count = 0; } void IncrementCounter() { count++; } void show() { cout<<count<<endl; } };</pre>	<pre>void main() { Test t1, t2, t3; t1.IncrementCounter(); t2.IncrementCounter(); t3.IncrementCounter(); t1.show(); t2.show(); t3.show(); }</pre>
--	---

31

Example – Static data members

<pre>class Test{ private: static int count; static const int y= 6; public: Test() { // count = 0; } void IncrementCounter() { count++; } void show() { cout<<count<<endl; } };</pre>	<pre>void main() { Test t1, t2, t3; t1.IncrementCounter(); t2.IncrementCounter(); t3.IncrementCounter(); t1.show(); t2.show(); t3.show(); }</pre>
---	---

int Test::count = 0;

32

Example – Static data members

<pre> class Test{ private: static int count; public: { count++; } static void show() { cout<<count<<endl; } }; int Test::count = 0; </pre>	<pre> void main() { Test t1, t2, t3; t1.IncrementCounter(); } </pre>
--	--

Static function of a class in C++ cannot access non-static variables, but, it can access static variable only. However, non-static member function can access static and non-static variable both.

33

Invoking Static Member Function - Syntax

Use ***ClassName::functionName(arguments)*** to invoke a static function

and ***ClassName::staticVariable*** to access static data

36

Instance or Static?

How do you decide whether a variable or function should be instance or static?

A variable or function that is dependent on (or related to) a specific instance of the class should be an instance variable or function.

A variable or function that is not dependent on (or not related to) a specific instance of the class should be a static variable or function.

Example: Every circle has its own radius. Radius is dependent on a specific circle. Therefore, radius is an instance variable of the Circle class. Since the getArea function is dependent on a specific circle, it is an instance function. Since numberOfObjects is not dependent on any specific instance, it should be declared static.

37

Immutable Objects and Classes

- If the **contents of an object cannot be changed** once the **object is created**, the **object is called an *immutable object*** and
- if a **class provides no setter or mutator functions**, then this class is called an ***immutable class***.

42

Immutable Classes

For a *class* to be *immutable*:

- It must **mark** all **data fields private**
- It **must not provide** any **mutator functions** and
- **No accessor functions** that would return a **reference or pointer to a mutable data fields**
- **Preventing changes** in **data members** from a member function by making **member function constant: *const function***

43

const Member Functions

- **A *const member function* guarantees that it will never modify any of its class's member data**

```
class Test
{
    private:
        int  alpha;
    public:
        void  nonFunc() //non-const member function
        { alpha = 99; } //OK
        void  conFunc() const //const member function
        { alpha = 99; }      //ERROR: can't modify a member
};
```

44

const Member Function - Example

```
class Distance
{
    private:
        int feet;
    public:
        Distance():feet(0)
        { }
        Distance(int ft): feet(ft)
        { }
        Distance addDist(Distance d)
        {
            Distance temp(5);
            temp.feet += d.feet;
            return temp;
        }
        void showDistance() const
        {
            cout<<"Feet = "<<feet;
        }
};
```

```
void main()
{
    Distance  dist1, dist3;
    Distance  dist2(11);
    dist3 = dist1.addDist(dist2);
    cout<<dist1.showDistance();
    cout<<dist2.showDistance();
    cout<<dist3.showDistance();
}
```

45

const Member Function – Example

```
class Alpha{
    private:
        int a;
        float b;
    public:
        Alpha():a(0),b(0.0)
        { }
        void func1( ) const
        {
            // a = 10; error: const function
            cout<<a;
        }
        void func2( ) const
        { cout<<b; }
};
```

```
void main() {
    Alpha al;

    al.func1();
    al.func2();
}
```

46

const Object

When an **object is declared as const**, you can't **modify it**.

It follows that **you can use only const member functions with it**, because they're the only ones that guarantee not to modify it.

47

const Object - Example

```
class Distance
{
    private:
        int feet; int inches;
    public:
        Distance():feet(0)
        { }
        Distance(int ft): feet(ft)
        { }
        int getDistance()
        {
            cout<<"Enter Distance";cin>>feet;
        }
        void showDistance() const
        {
            cout<<"Feet = "<<feet;
            cout<<"Inches="<<inches;
        }
};
```

```
void main()
{
    const Distance dist1;

    //cout<<dist1.getDistance();
    // Error: getDistance not constant
    cout<<dist1.showDistance();
}

//showDistance() is ok because it is
constant function
```

48

Difference between *class* and *struct*

By default, all data fields of a struct are public. However, by default, all data fields of a class are private.

The *struct* type defined in (a) can be replaced by the *class* defined in (b) by using *public* access modifier.

```
struct Student
{
    int id;
    char firstName[30];
    char mi;
    char lastName[30];
};
```

(a)

```
class Student
{
public:
    int id;
    char firstName[30];
    char mi;
    char lastName[30];
};
```

(b)